GridFront

Scout Ecosystem Technical Overview

How Scout uses OAK-D stereo spatial detection, an Android tablet, and local JSON messages to detect people around heavy equipment.

Version: Scout v0.1.0, schema 1.

Updated: May 10, 2026.

What Scout Is

Scout is GridFront's mounted spatial detection ecosystem for mobile plant: excavators, wheel loaders, dozers, dump trucks, graders, and similar heavy equipment. It uses OAK-D stereo cameras to detect people, measure where they are in machine-relative space, classify the active zone, and present that state to the operator on a tablet.

The product category aligns with human-form recognition camera systems used in construction safety: multi-camera coverage, configurable inner and outer detection zones, visual/audible operator alerts, and system health awareness. Scout's implementation goes deeper by keeping spatial perception on the OAK-D and using the tablet as the local setup, display, and orchestration surface.

Perception

Runs the flashed standalone pipeline, tracks objects, transforms detections into the machine frame, and classifies each detection so every downstream surface sees the same safety state.

→

Local Runtime

Receives UDP packets, keeps the freshest state, serves the local API, and owns setup for machine, camera, zone, model, and runtime settings.

→

Operator Surface

Renders radar, status, alarms, settings, and camera health from the tablet's local HTTP/SSE API without needing a cloud round trip.

State Codes

Scout uses numeric state codes for decisions and simple text labels for readability. The code is the decision value: higher numbers mean higher urgency, so displays and alert logic can compare `3 > 2 > 1 > 0` without parsing words. The string label stays beside the code so logs, demos, and support conversations remain readable.

Code	Scene meaning	Per-detection meaning	UI behavior
0	No detections	Not used on a detection object	No detection state
1	Detection exists, outside zones	Detection outside configured zones	Track object, no alert
2	At least one yellow-zone detection	Detection in yellow zone	Yellow visual state and chime
3	At least one red-zone detection	Detection in red zone	Red visual state and repeating alarm

> Code 0 belongs to the scene, not to an individual detection. No detections means the array is empty. A detection object starts at code 1 because the object exists, even when it is outside the configured zones.

Channels

Channel	Direction	Purpose
`5556/UDP`	OAK-D to tablet	Live detection packets.
`5557/UDP`	OAK-D both ways with tablet	Camera config request and tablet config push.
`127.0.0.1:8080`	WebView to tablet app	Local app, local API, and live SSE stream.
Display UDP	Optional backend to cab displays	Trimmed latest-state packets for display firmware.

Multi-Camera Operation

The Scout contract is designed so one tablet can build a single safety view from multiple OAK-D cameras. The target architecture supports up to six cameras streaming to the same tablet and appearing together in one operator UI.

How cameras stay separate

Each OAK-D has its own `camera_id`, pose, IP address, and config entry. Every detection packet includes that `camera_id`.

How the UI becomes one view

The tablet receives packets on the same UDP channel, keeps the freshest packet per camera, and the WebView merges them into one radar, count, closest-distance readout, and scene state.

Aggregate field	Multi-camera behavior
`scene_state_code`	The UI uses the highest active code across all cameras. Red on any camera makes the scene red.
`detection_count`	Counts detections from all active cameras after packets are merged.
`closest_m`	Uses the closest detection reported by any camera.
`zone_code`	Stays attached to each detection, so individual dots can still be colored correctly.

Detection Packet

The detection packet is the main live message. It describes what the camera sees now, not an event ledger. If a UDP packet is lost, the next packet replaces it; if the link goes stale, the UI should clear or show an offline state rather than replay old detections.

Sent by

OAK-D standalone pipeline

Received by

Android tablet UDP listener

Transport

`UDP :5556`

Important fields

`scene_state_code` drives overall UI state.

`zone_code` drives each detection's display color and alarm behavior.

`x_m` and `y_m` are machine-frame meters.


```
{
  "schema_version": 1,
  "type": "detections",
  "camera_id": "cam-0",
  "scene_state_code": 3,
  "scene_state": "red",
  "detections": [
    {
      "track_id": 12,
      "label": "person",
      "x_m": 0.72,
      "y_m": 5.18,
      "distance_m": 4.2,
      "zone_code": 3,
      "zone": "red",
      "zone_id": "red-inner"
    }
  ],
  "summary": {
    "detection_count": 1,
    "outside_count": 0,
    "yellow_count": 0,
    "red_count": 1,
    "highest_zone_code": 3,
    "closest_m": 4.2,
    "raw_count": 1
  },
  "units": "m",
  "link": "ok",
  "ts": 1778371200.123
}
```

No-Detection Packet

No detections are represented by an empty `detections` array and `scene_state_code: 0`. This is different from a detection that exists outside the monitored zones, which uses code 1.


```
{
  "schema_version": 1,
  "type": "detections",
  "camera_id": "cam-0",
  "scene_state_code": 0,
  "scene_state": "none",
  "detections": [],
  "summary": {
    "detection_count": 0,
    "outside_count": 0,
    "yellow_count": 0,
    "red_count": 0,
    "highest_zone_code": 0,
    "closest_m": null
  },
  "units": "m",
  "link": "ok",
  "ts": 1778371200.123
}
```

Config Sync

The tablet owns the active machine/camera/zone configuration. The OAK-D asks for config on boot and every 30 seconds, and the tablet also pushes config immediately after camera-facing edits. This keeps setup centralized on the operator tablet while letting each camera run its perception pipeline independently.

Request from camera

```
{
  "type": "config_request",
  "camera_id": "cam-0"
}
```

Response from tablet

```
{
  "type": "config",
  "camera_id": "cam-0",
  "pose": {
    "x_m": 0.0,
    "y_m": 4.0,
    "yaw_deg": 0.0
  },
  "zones": [
    {
      "id": "red-inner",
      "label": "red",
      "severity_code": 3,
      "r_m": 3.5
    },
    {
      "id": "yellow-outer",
      "label": "yellow",
      "severity_code": 2,
      "r_m": 6.0
    }
  ],
  "machine_footprint_m": {
    "length": 8.0,
    "width": 2.5
  },
  "target_fps": 10.0
}
```

Tablet Local API

The WebView reads from the tablet over localhost. This keeps the UI fast and lets Scout run without an external server.

Endpoint	Purpose	Pushes camera config?
`GET /api/spatial`	Latest detection packet.	No
`GET /api/spatial/stream`	Live SSE stream of detection packets.	No
`GET /api/config`	Full tablet-owned config.	No
`POST /api/config`	Replace full config.	Yes, all cameras
`POST /api/zones`	Replace red/yellow zone distances.	Yes, all cameras
`POST /api/cameras/{id}/pose`	Update one camera mount pose.	Yes, that camera
`POST /api/runtime`	Update runtime settings such as `target_fps`.	Yes, when FPS changes

How Product Should Think About It

Not an event ledger

Detection packets describe what Scout sees now. Lost packets are replaced by the next packet, and stale links should clear the scene.

Safety classification

The OAK-D classifies detections before the tablet displays them. The UI presents one consistent camera-authored safety state.

Setup truth

The tablet stores machine, camera, zone, model, and runtime config, then pushes the camera-facing subset.

Integration Rules

Rule	Why it matters
Use `scene_state_code` for the overall UI/display state.	It gives one unambiguous decision value for red, yellow, outside, or none.
Use `zone_code` for individual detections.	It lets displays color each dot without parsing text.
Treat `detections: []` as no detections.	Do not infer no detections from a per-detection value.
Keep strings for readability, not decisions.	Strings help logs and demos, but numbers prevent casing and wording bugs.
Honor `link: "stale"`.	Stale data should clear or show offline state rather than freeze old detections.